

Лабораторная работа 4

Исаев Р. К.

Российский университет дружбы народов, Москва, Россия

Информация

- Исаев Рамазан Курбанович
- студент НКАБД 01-24
- Российский университет дружбы народов
- https://github.com/aksa077/study_2024-2025_os-intro.git

Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Задание

Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами: `-i`inputfile — прочитать данные из указанного файла; `-o`outputfile — вывести данные в указанный файл; `-r`шаблон — указать шаблон для поиска; `-C` — различать большие и малые буквы; `-n` — выдавать номера строк. а затем ищет в указанном файле нужные строки, определяемые ключом `-r`. Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в `o` коде завершения в оболочку. Команд- ный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено. Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до `N` (например 1.tmp, 2.tmp, 3.tmp,4.tmp и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же ко- мандный файл должен уметь удалять все созданные им файлы (если они существуют). Написать командный файл, который с помощью команды `tar` запаковывает в архив все

Теоретическое введение

Bash (Bourne Again Shell) — это мощная командная оболочка Unix, которая используется для выполнения различных задач в терминале. Bash предоставляет интерактивный интерфейс, в котором пользователи могут вводить команды, а затем получать результаты. Она также поддерживает скрипты оболочки, которые представляют собой текстовые файлы, содержащие последовательность команд Bash для автоматизации задач. Bash широко используется в средах Unix и Linux, а также поддерживается Windows с помощью подсистемы Windows для Linux (WSL). Перечислим основные возможности этой оболочки.

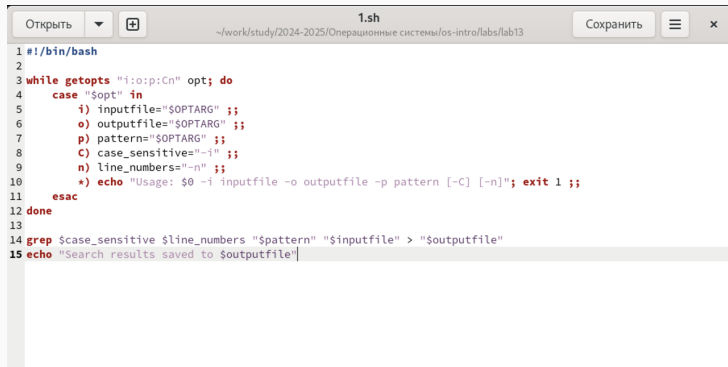
Обработка команд. Bash может обрабатывать как простые, так и сложные команды. Простые состоят из одного действия и, возможно, некоторых аргументов. Сложные команды могут содержать несколько простых, объединенных с помощью операторов конвейера (`|`), перенаправления ввода и вывода (`<`, `>`, `>>`), условных операторов (`if/else`, `case/esac`, `while/do`). О них мы расскажем позже.

Расширенный ввод, редактирование строк. Оболочка предоставляет функции расширенного ввода, такие как автодополнение, которое предлагает возможные варианты завершения команд и имен файлов по мере их ввода. Она также поддерживает

Выполнение лабораторной работы

Выполнение лабораторной работы

Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами: `-i`inputfile — прочитать данные из указанного файла; `-o`outputfile — вывести данные в указанный файл; `-r`шаблон — указать шаблон для поиска; `-C` — различать большие и малые буквы; `-n` — выдавать номера строк. а затем ищет в указанном файле нужные строки, определяемые ключом `-p`.



```
1 #!/bin/bash
2
3 while getopts "i:o:p:Cn" opt; do
4     case "$opt" in
5         i) inputfile="$OPTARG" ;;
6         o) outputfile="$OPTARG" ;;
7         p) pattern="$OPTARG" ;;
8         C) case_sensitive="-i" ;;
9         n) line_numbers="-n" ;;
10        *) echo "Usage: $0 -i inputfile -o outputfile -p pattern [-C] [-n]"; exit 1 ;;
11    esac
12 done
13
14 grep $case_sensitive $line_numbers "$pattern" "$inputfile" > "$outputfile"
15 echo "Search results saved to $outputfile"
```



Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку. Команд- ный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено.

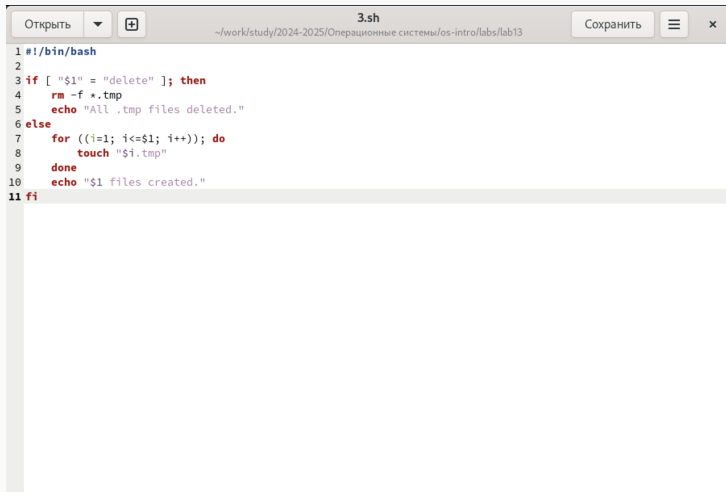


```
2.sh
~/work/study/2024-2025/Операционные системы/os-intro/labs/lab13
Открыть  + Сохранить  x

1 #!/bin/bash
2
3 cat << EOF > check_number.c
4 #include <stdio.h>
5 #include <stdlib.h>
6
7 int main() {
8     int num;
9     printf("Enter a number: ");
10    scanf("%d", &num);
11    if (num > 0) exit(1);
12    if (num < 0) exit(2);
13    exit(0);
14 }
15 EOF
16
17 gcc check_number.c -o check_number
18 ./check_number
19 status=$?
20 if [ $status -eq 1 ]; then
21     echo "The number is positive."
22 elif [ $status -eq 2 ]; then
23     echo "The number is negative."
24 else
25     echo "The number is zero."
26 fi
```



Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до [?] (например 1.tmp, 2.tmp, 3.tmp, 4.tmp и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют).

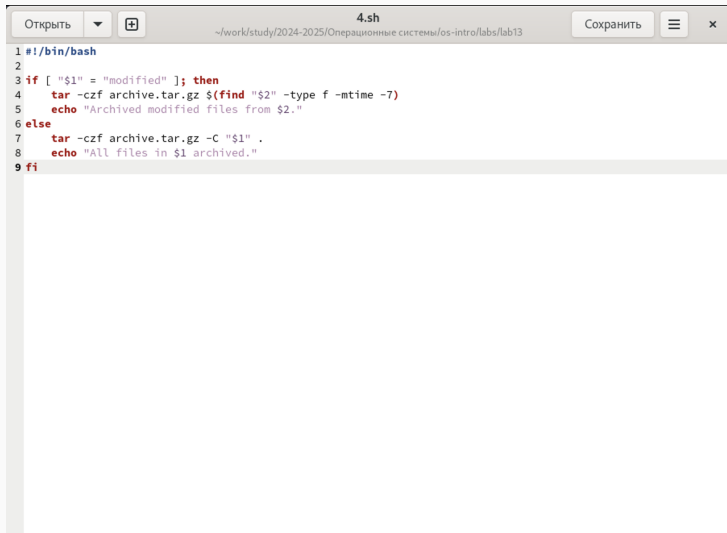


```
3.sh
~/work/study/2024-2025/Операционные системы/os-intro/labs/lab13
Открыть + Сохранить ☰ x

1 #!/bin/bash
2
3 if [ "$1" = "delete" ]; then
4     rm -f *.tmp
5     echo "All .tmp files deleted."
6 else
7     for ((i=1; i<=$1; i++)); do
8         touch "$i.tmp"
9     done
10    echo "$1 files created."
11 fi
```



Написать командный файл, который с помощью команды tar запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду find)



```
1 #!/bin/bash
2
3 if [ "$1" = "modified" ]; then
4     tar -czf archive.tar.gz $(find "$2" -type f -mtime -7)
5     echo "Archived modified files from $2."
6 else
7     tar -czf archive.tar.gz -C "$1" .
8     echo "All files in $1 archived."
9 fi
```

Контрольные вопросы

Контрольные вопросы

Каково предназначение команды `getopts`? Осуществляет синтаксический анализ командной строки, выделяя флаги, и используется для объявления переменных. Синтаксис команды следующий: `getopts option-string variable`. Флаги – это опции командной строки, обычно помеченные знаком минус; Например, `-F` является флагом для команды `ls -F`. Иногда эти флаги имеют аргументы, связанные с ними. Программы интерпретируют эти флаги, соответствующим образом изменяя свое поведение. Строка опций `option-string` — это список возможных букв и чисел соответствующего флага. Если ожидается, что некоторый флаг будет сопровождаться некоторым аргументом, то за этой буквой должно следовать двоеточие. Соответствующей переменной присваивается буква данной опции. Если команда `getopts` может распознать аргумент, она возвращает истину. Принято включать `getopts` в цикл `while` и анализировать введенные данные с помощью оператора `case`. Предположим, необходимо распознать командную строку следующего формата: `testprog -ifile_in.txt -ofile_out.doc -L -t -r`. Вот как выглядит использование оператора `getopts` в этом случае: `while getopts o:i:Ltr optletter do case optletter in) o flag = 1; oval =OPTARG;; i) iflag=1; ival=$OPTARG;; L) Lflag=1;;` ^{10/12}

Выводы

Мы изучили основы программирования в оболочке ОС UNIX. Научились писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Список литературы
